

Data Transmission Stability Testing of DHT22 Sensor and OTA Firmware Update Mechanism on ThingsBoard Platform Using ESP32-S3

Muhammad Aufa Affandi^a, Muhammad Salman Alfarisyi^a, Ahmad Radhy^a

^a*Department of Instrumentation Engineering Technology,
Faculty of Vocational Studies, Institut Teknologi Sepuluh Nopember (ITS),
Jl. Raya ITS, Keputih, Sukolilo, Surabaya 60111, Indonesia
2042231011@student.its.ac.id
2042231006@student.its.ac.id*

Abstract

This study aims to analyze the stability of data transmission and the implementation of an Over-The-Air (OTA) firmware update mechanism on an Internet of Things (IoT) monitoring system using the ESP32-S3 microcontroller and the DHT22 sensor. The system was developed to measure temperature and humidity in real time and transmit data to the ThingsBoard Cloud platform through the MQTT protocol. The experimental setup involved testing data transmission reliability, evaluating network latency, and performing OTA firmware updates remotely. The device was programmed using Rust Embedded with a no-std architecture for optimized performance and memory efficiency. The experimental results show that the system successfully transmitted telemetry data continuously with an average network latency of approximately 1.4 seconds, indicating stable and efficient MQTT communication. The OTA update process also ran smoothly, allowing firmware upgrades without physical access to the device. Overall, this work demonstrates a reliable IoT framework that combines real-time data monitoring and remote firmware maintenance for efficient device management.

Keywords: Internet of Things, ESP32-S3, DHT22 sensor, MQTT protocol, ThingsBoard Cloud, Over-The-Air (OTA), data transmission stability, latency analysis.

I. INTRODUCTION

The Internet of Things (IoT) has emerged as one of the most transformative technologies in modern industrial and research domains, enabling seamless connectivity between sensors, devices, and cloud platforms for real-time data acquisition and analysis [1], [2]. In monitoring systems, data stability and reliability are critical factors that determine the accuracy and effectiveness of information exchange between devices and servers. However, challenges such as inconsistent data transmission, limited connectivity, and manual firmware updates often reduce system efficiency [3], [4]. The ESP32-S3 microcontroller offers a powerful and cost-effective platform for IoT development, featuring integrated Wi-Fi, Bluetooth connectivity, and sufficient computational resources for embedded applications [5]. Combined with the DHT22 sensor, which provides temperature and humidity measurements, this platform can serve as a foundation for building reliable IoT monitoring systems [6], [7]. In addition, implementing an Over-The-Air (OTA) firmware update mechanism enables remote system maintenance without the need for physical access, enhancing the scalability and long-term usability of IoT devices [8], [9]. To ensure optimal performance, this research also explores the use of the Rust programming language with an embedded no-std environment, which provides memory safety, concurrency, and efficient execution for microcontroller-based applications [10], [11]. Therefore, this study focuses on developing and testing an IoT monitoring system that utilizes the ESP32-S3 and DHT22 sensors to evaluate data transmission stability and the OTA firmware update process through the MQTT protocol on

the ThingsBoard Cloud platform. The results of this research are expected to contribute to the design of more robust and maintainable IoT systems with enhanced reliability in data communication and firmware management.

II. LITERATURE STUDY

A. State of the Art

1) The Development of the Internet of Things (IoT) in Monitoring Systems

The Internet of Things (IoT) has rapidly evolved, particularly in monitoring applications, enabling sensors and devices to automatically collect and exchange data in real time. Al-Fuqaha et al. [1] described IoT as a convergence of sensor technology, communication protocols, and network architectures that form the foundation for modern monitoring systems.

To address challenges such as latency and network load, recent research trends emphasize the integration of edge computing with IoT. Studies such as “Integrating Edge Computing and IoT for Real-Time Air and Water Quality Monitoring Systems” [12] show that processing data near the source reduces communication delays and network bandwidth usage. Similarly, Roostaei et al. [13] proposed a hybrid IoT–edge architecture that improves monitoring system efficiency. Peng [14] further integrated multimodal sensors and edge intelligence, enabling more responsive and energy-efficient environmental monitoring.

In summary, IoT-based monitoring has shifted beyond real-time data collection toward hybrid architectures that combine local data processing and

cloud analytics for faster and more stable system performance.

2) Implementation of MQTT and Cloud Platforms for Data Visualization

In modern IoT systems, lightweight protocols such as MQTT play a crucial role in reliable real-time data transmission and visualization. Nitol Saha et al. [15] designed an industrial energy and environment monitoring system using MQTT and a cloud-based dashboard to analyze device performance remotely. Shvaika et al. [16] discussed the scalability of MQTT through a distributed broker architecture (TBMQ) that supports high client loads without compromising latency.

Montesclaros et al. [17] applied MQTT with visualization techniques to identify power consumption patterns in smart buildings, while El-Basioni [18] proposed a conceptual model describing MQTT topic structures, QoS configurations, and data flow across IoT–Cloud systems. These studies highlight MQTT's reliability in real-time visualization and decision support through cloud dashboards.

3) DHT22 Sensor for Temperature and Humidity Monitoring

The DHT22 sensor is widely used in IoT monitoring systems due to its ability to measure temperature and humidity with good precision at a low cost. Ahmad et al. [9] evaluated DHT22's accuracy compared to industrial instruments and concluded it is sufficiently reliable for everyday IoT applications. Yulizar et al. [10] compared DHT22 with DHT11 and DS18B20, showing DHT22's superior consistency in common environments.

Putri et al. [19] successfully implemented DHT22 on an ESP32 prototype that transmitted temperature and humidity data to the ThingSpeak platform, achieving stable operation under different room conditions. Similarly, Wardani et al. [20] confirmed that DHT22 can deliver consistent readings when deployed in greenhouse environments, validating its suitability for multi-sensor applications.

4) IoT Data Transmission Stability and Network Performance

The stability of data transmission and network performance is crucial in IoT systems to ensure that sensor data is delivered accurately and on time. Puthiyidam and Joseph [21] studied the impact of message size and client load on network latency, showing that smaller payloads and fewer connections reduce delay significantly.

Amirkhanov et al. [22] compared MQTT over TCP, MQTT over WebSocket, and HTTP for digital twin systems, finding that MQTT over TCP offers the lowest average latency. Palmese et al. [23] also discussed adaptive QoS control for MQTT-SN to improve reliability under dynamic network conditions. These findings confirm that protocol selection and QoS configuration are key to achieving stable and efficient IoT communication.

5) Firmware Update Using Over-The-Air (OTA) Mechanisms

Over-The-Air (OTA) firmware updates enable IoT devices to receive software upgrades remotely without physical access. El Jaouhari and Bouvet [4] identified OTA as a cost-effective and scalable maintenance method. Palmese et al. [23] emphasized security during firmware transfer through encryption and digital signature verification.

Park et al. [8] developed a lightweight OTA system optimized for low-memory devices, while Malumbres et al. [7] explored LoRa-based OTA for simultaneous updates to multiple devices. Wei et al. [24] proposed incremental OTA updates, transmitting only modified firmware segments to improve efficiency.

Overall, OTA mechanisms ensure flexibility and maintainability in large-scale IoT deployments such as ESP32-based monitoring systems.

6) Research Gap and Relevance of This Study

Previous studies have primarily focused on improving OTA speed or security, often neglecting energy efficiency and data transmission stability under real network constraints. Few works combine OTA, MQTT optimization, and real-time monitoring on the ESP32-S3 platform.

This study is relevant as it develops an efficient, secure, and easily deployable OTA mechanism for ESP32-S3-based IoT devices while analyzing the stability of MQTT data transmission during updates. The project aims to provide a more adaptive and practical framework for modern wireless IoT monitoring systems.

B. Theoretical Background

1) Internet of Things (IoT)

The IoT concept connects various physical devices to the Internet, allowing autonomous data exchange through sensors and wireless communication technologies such as Wi-Fi or Bluetooth [1], [2]. In this project, IoT principles are implemented to connect the ESP32-S3 microcontroller with the DHT22 sensor for remote environmental monitoring using the MQTT protocol and ThingsBoard Cloud [3].

2) ESP32-S3 Microcontroller

The ESP32-S3, developed by Espressif Systems, is a dual-core Xtensa LX7 processor with AI acceleration, Wi-Fi, and Bluetooth 5.0 connectivity, designed for high-performance and low-power IoT applications [3], [5]. It also includes security features such as flash encryption and secure boot, making it suitable for OTA firmware updates and real-time cloud monitoring [4].

3) Ubuntu as IoT Development Environment

Ubuntu, a Linux-based open-source operating system, offers a stable and flexible development environment for IoT systems [25]. It supports programming tools like Rust and C++, MQTT brokers, and OTA deployment through open-source packages such as Mosquitto, making it ideal for integrated testing and maintenance.

4) Firmware in IoT Systems

Firmware is low-level software embedded in microcontrollers to handle communication, data

processing, and device control [26]. Modern IoT firmware follows modular and updatable design principles, allowing scalability and efficient OTA updates.

5) Over-The-Air (OTA) Update Mechanism

OTA updates enable wireless firmware upgrades. Studies emphasize the need for lightweight encryption, memory optimization, and incremental updates to ensure secure and reliable processes in constrained devices [4], [7], [8].

6) DHT22 Sensor

The DHT22 sensor measures temperature and humidity with accuracy up to $\pm 0.5^\circ\text{C}$ and $\pm 2\%$ RH. Its digital one-wire communication makes it suitable for integration with microcontrollers like ESP32 [9], [10].

7) MQTT Protocol

MQTT (Message Queuing Telemetry Transport) is a lightweight publish-subscribe communication protocol widely used in IoT due to its efficiency in low-bandwidth networks [15], [16]. It supports different Quality of Service (QoS) levels, ensuring reliable telemetry delivery between devices and servers.

8) Platform Cloud ThingsBoard

ThingsBoard is an open-source IoT platform for device management, data storage, and real-time visualization [3], [16]. It also supports OTA management through firmware repositories and REST APIs, providing seamless integration with IoT devices.

9) Programming Languages for Embedded Systems (Rust/C++)

C++ remains a standard for embedded systems due to its efficiency and low-level control [9]. However, Rust offers memory safety and concurrency without sacrificing performance, making it a promising alternative for modern IoT applications [10].

III. METHODOLOGY

A. System Design and Architecture

The proposed IoT system was designed to perform real-time temperature and humidity monitoring using a DHT22 sensor connected to an ESP32-S3 microcontroller. The device was programmed using the Rust Embedded (bare-metal) environment, providing memory efficiency and secure hardware resource access.

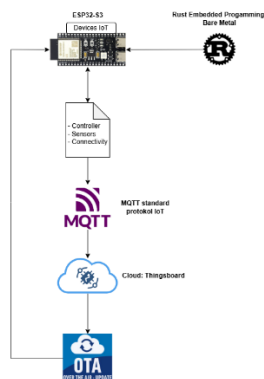


Figure 3.1 System Design and Architecture

As illustrated in Fig. 3.1, the ESP32-S3 acts as the central processing unit, periodically acquiring data from the DHT22 sensor, processing the readings, and transmitting them to the ThingsBoard Cloud platform through the MQTT protocol. The transmitted data includes temperature, humidity, and timestamp values obtained from the Real-Time Clock (RTC) module for latency and reliability analysis.

The MQTT protocol was selected due to its lightweight, reliable, and low-bandwidth characteristics. Additionally, the system supports Over-The-Air (OTA) firmware updates, enabling remote software upgrades without physical access to the device. To ensure communication security, MQTT authentication is configured using ThingsBoard's token-based access. This combination of features allows the system to be flexible, secure, and easily maintainable.

B. Hardware Design

The hardware components of the system include the ESP32-S3 microcontroller as the main processing unit, a DHT22 temperature and humidity sensor, and supporting modules such as jumpers and a 5V power supply. The ESP32-S3 features integrated Wi-Fi and Bluetooth, a dual-core Xtensa LX7 processor, and energy-efficient operation suitable for IoT applications.

The DHT22 sensor connects to the ESP32-S3 through a digital GPIO4 pin, with VCC connected to 3.3V and GND to ground. The ESP32-S3 reads the sensor data, appends a timestamp using its internal RTC, and transmits the readings to the ThingsBoard Cloud through MQTT. The entire circuit is assembled on a breadboard for easy testing and modification during development (Fig. 3.2).

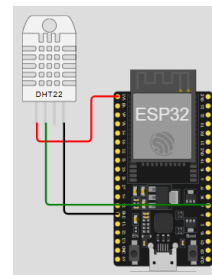


Figure 3.2 Wiring between ESP32-S3 and DHT22

C. Software Development

The system software was developed in the Rust Embedded language running in bare-metal mode on the ESP32-S3. The main functions include system initialization, sensor data acquisition, MQTT communication, and OTA firmware update handling (Fig. 3.3).

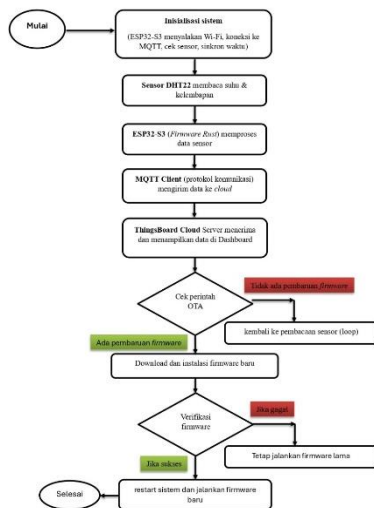


Figure 3.3 Software workflow of the ESP32-S3-based IoT system

During initialization, the ESP32-S3 activates Wi-Fi connectivity, synchronizes time via NTP, and verifies the availability of the DHT22 sensor. The collected data are encoded in JSON format, containing:

- send_time: local device time when the data are sent
- ts: system timestamp in milliseconds
- temperature: measured temperature in °C
- humidity: measured humidity in %

The JSON payload is published to the MQTT topic `v1/devices/me/telemetry` every 60 seconds. The device terminal logs indicate successful transmission with a “data sent” message.

The firmware also implements an OTA update mechanism, allowing the device to automatically detect, download, and install new firmware versions from the ThingsBoard repository. If the update process fails, the system retains the previous firmware to maintain operational stability.

D. System Integration and Visualization

The Rust project configuration file (`Cargo.toml`) defines the dependencies and build profiles required for compilation on the ESP32-S3. The essential libraries include:

- `esp-idf-svc` for Wi-Fi, MQTT, and system management,
- `dht-sensor` for DHT22 data acquisition,
- `serde_json` for JSON formatting,
- `chrono` for time and timestamp management, and
- `anyhow` and `log` for debugging and logging.

The ThingsBoard Cloud dashboard (Fig. 3.4) displays real-time temperature and humidity values in both time-series and tabular formats. The blue and green lines on the graph represent temperature and humidity trends, respectively, while the telemetry table shows timestamp, temperature, humidity, and firmware state values.



Figure 3.4 Monitoring Dashboard Design on ThingsBoard Cloud

E. Data Visualization and Latency Analysis

For experimental analysis, data were exported from the ThingsBoard Cloud and visualized using Gnuplot on Ubuntu. The CSV files were processed to generate time-series plots of temperature and humidity, as well as latency measurements between data transmission and reception.

The following Gnuplot commands were used to display the temperature-humidity trend and latency distribution (Fig. 3.5 & Fig. 3.5.1):

The screenshot shows a terminal window titled "Pace@Pace~". The prompt is "Pace@Pace~:". Below the prompt, there are three tabs labeled "Pace@Pace~:", "Pace@Pace~:", and "Pace@Pace~:". The first tab is active and displays the following MATLAB code:

```
% Immediate help type "help" (plot window: hlt 'h')

% Terminal file is now at
gnuplot> set datarfile separator ';'
gnuplot> set wstate time
gnuplot> set ttimefmt "%Y-%m-%d %H:%M:%S"
gnuplot> set format x "%Y-%m-%d %H:%M:%S" # tampilan tanggal dan jam di baris b
gnuplot> set title "Sensor Data from ThingsBoard"
gnuplot> set xlabel "Time"
gnuplot> set ylabel "Temperature (A°C)"
gnuplot> set y2label "Humidity (K%)
gnuplot> set grid
gnuplot> set key left top
gnuplot> set ytics nonerror
gnuplot> set y2tics
gnuplot>
gnuplot> % Gunakan dua sumbu Y (kiri=Temperature, kanan=Humidity)
gnuplot> plot '+' title "x 2 sensor_data.csv" using 1:3 with Lines lw 2 title "Te
gnuplot> temperature (A°C)" axis y1,
gnuplot> "x 2 sensor_data.csv" using 1:4 with Lines lw 2 title "Humidity (K
gnuplot> %"
gnuplot> % SocketHandler can only be used with threads started with QThread
gnuplot>
```

Figure 3.5 Gnuplot Commands for Sensor Data Visualization

```
gnuplot> set key top left
gnuplot> plot "latency_ready.csv" using 1:2 with linespoints lw 2 lc rgb "orange"
title "Latency"
warning: Skipping data file with no valid points
x
x range is invalid
gnuplot> head -n 5 latency_ready.csv | cat -\A
\A
invalid command

gnuplot> reset
gnuplot> set datafile separator ";"
gnuplot> set decimalsign locale
decimal_sign in locale is
gnuplot> set title "Latency Analysis: Sensor RTC vs Thingsboard"
gnuplot> set xlabel "Sample Index"
gnuplot> set ylabel "Latency (seconds)"
gnuplot> set grid
gnuplot> set key top left
gnuplot> plot "latency_ready.csv" using 1:2 with linespoints lw 2 lc rgb "orange"
title "Latency"
gnuplot> s
```

Figure 3.5.1 Gnuplot Commands for Latency Graph

Latency analysis utilized the `latency_ready.csv` dataset to calculate the time delay between sensor data acquisition and server reception. The analysis process was conducted using Gnuplot on Ubuntu to visualize latency trends and evaluate transmission consistency under various network conditions.

IV. RESULTS AND DISCUSSION

A. System Implementation Results

The IoT monitoring system was successfully implemented using the ESP32-S3 microcontroller, DHT22 sensor, and a 5V power adapter as the primary power source. The microcontroller was programmed in Rust and connected to a laptop via USB for firmware flashing.

After executing the cargo build and cargo flash commands, the terminal displayed several key logs, including:

- “Connected to Wi-Fi” – indicating successful Wi-Fi connectivity.
- “MQTT broker connected” – confirming the device connection to the ThingsBoard Cloud server.
- “Data sent (humidity, temperature)” – verifying successful data transmission via MQTT.

Additionally, the terminal log confirmed that the device could perform remote firmware updates through the OTA mechanism. These results demonstrate that the system operated stably, with all communication and update processes executed without error. (See Fig. 4.1 for device setup and Fig. 4.1.1 for terminal connectivity output.)

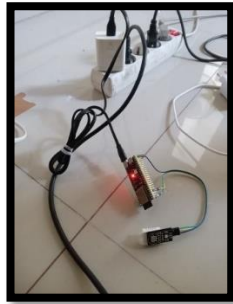


Figure 4.1 ESP32-S3 is connected to the power adapter and the DHT22 sensor

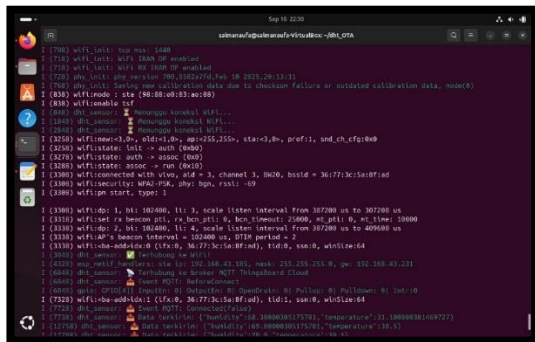


Figure 4.1.1 Connectivity display in the Ubuntu terminal

B. Sensor Data Transmission Performance

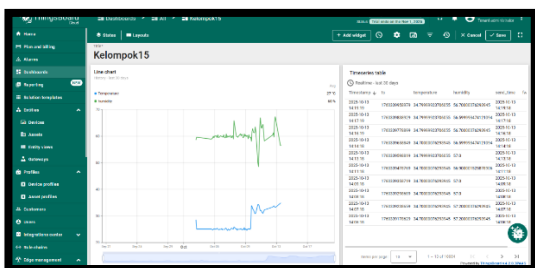


Figure 4.2 The ThingsBoard Cloud dashboard displays temperature and humidity graphs

The sensor transmission test evaluated the ability of the ESP32-S3 and DHT22 to measure and transmit temperature and humidity data to the ThingsBoard Cloud in real time using the MQTT protocol. Each JSON payload contained:

- Temperature (°C),
- Humidity (%), and
- A timestamp generated by the internal RTC of the ESP32-S3.

The ThingsBoard dashboard displayed the collected data in real time, with time-series graphs visualizing the variation of temperature and humidity over time (Fig. 4.2).

Data acquisition was conducted from 2–13 October 2025, during which the system consistently transmitted telemetry to the server with minimal delay. Some interruptions occurred during power configuration and system calibration (e.g., 3–4 October), but these were resolved after switching to a 5V external adapter for 24-hour operation.

By 9–12 October 2025, both send_time (device RTC) and ts (server timestamp) parameters were successfully synchronized, enabling precise latency analysis. Overall, the device maintained reliable and stable transmission performance throughout the testing period.

C. OTA Firmware Update Validation

The OTA update test was conducted to verify the ability of the system to perform remote firmware upgrades through the ThingsBoard Cloud platform. A new firmware version was uploaded to the server repository and automatically distributed to the ESP32-S3 via MQTT, following the configured OTA attributes.

During the update process, the device downloaded, verified, and installed the firmware while reporting telemetry parameters such as fw_state, fw_version, target_fw_version, and update_tag (see Fig. 4.3 and Fig. 4.3.1).

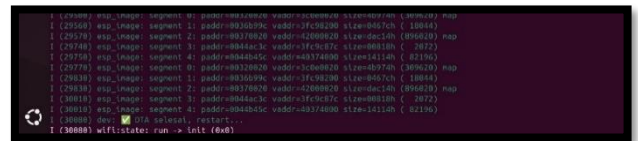


Figure 4.3 Terminal log display upon successful OTA update

Last update time	Key	Value
2025-10-12 13:10:03	fw_state	IDLE
2025-10-12 13:10:02	fw_version	PicoP3-v3.0
2025-10-12 13:10:02	humidity	56.700007629945
2025-10-12 14:19:19	send_time	2025-10-12 14:19:19
2025-10-09 17:02:23	target_fw_tag	Update-OTA-63.2.0
2025-10-09 17:02:23	target_fw_title	Update-OTA-63
2025-10-09 17:02:23	target_fw_ts	176004203892
2025-10-09 17:02:23	target_fw_version	2.0
2025-10-12 14:19:19	temperature	34.79999923760055
2025-10-12 14:19:19	ts	1760239595979

Figure 4.3.1 OTA parameter display on ThingsBoard Cloud

Upon successful installation, the terminal displayed the message “OTA update completed, restarting...”, and the device rebooted automatically to load the new firmware version (fw_version = PaceS3-v2.0). The final status fw_state = IDLE confirmed that the update completed successfully without communication errors.

These results validate that the OTA process was stable and reliable under normal Wi-Fi network conditions.

D. Data Visualization and Latency Analysis

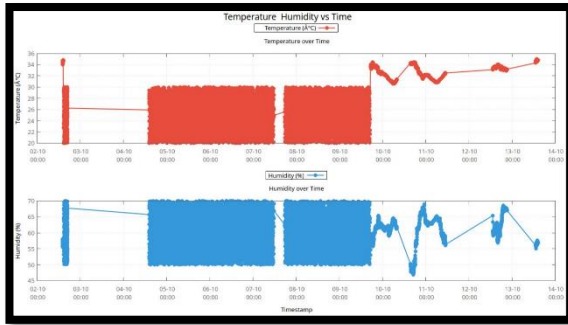


Figure 4.4 Temperature and Humidity vs Time Graph using Gnuplot

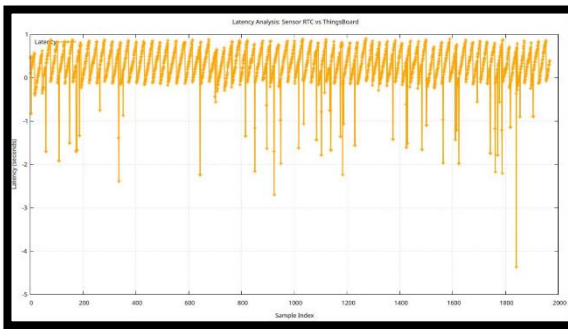


Figure 4.4.1 Latency Analysis Graph: Sensor RTC vs ThingsBoard

Data visualization and latency analysis were performed using Gnuplot on Ubuntu. Sensor data exported from the ThingsBoard platform were plotted in time-series graphs to observe environmental trends and network delay characteristics.

Fig. 4.4 illustrates the temperature and humidity variation recorded between 2–13 October 2025, where the temperature ranged from 20°C to 36°C and humidity from 45% to 70%. The data gap observed on 3 October corresponded to the temporary power test phase using a 5V adapter for continuous operation.

Latency analysis (Fig. 4.4.1) was based on the difference between send_time (ESP32-S3 RTC) and ts (server timestamp). The calculated average latency was approximately 1.4 seconds, with minor fluctuations caused by Wi-Fi network variability and MQTT reconnection cycles.

The latency remained within acceptable limits for IoT monitoring systems, demonstrating that the MQTT communication between the ESP32-S3 and ThingsBoard Cloud operated with high reliability and negligible packet loss. These findings confirm that the system achieved consistent real-time performance suitable for cloud based IoT applications.

V. CONCLUSION

Based on the design, implementation, and testing results of the IoT-based temperature and humidity monitoring system using the ESP32-S3 microcontroller, DHT22 sensor, and ThingsBoard Cloud platform with the MQTT protocol, it can be concluded that the system performed successfully and reliably. The device was able to transmit temperature and humidity data in real time to the cloud in JSON format, including temperature, humidity, and timestamp values from the ESP32-S3’s internal RTC.

Experimental evaluation showed that communication between the device and the server remained stable throughout the test period (2–13 October 2025) with no data loss and consistent network connectivity. Latency analysis using Gnuplot indicated an average delay of approximately 1.4 seconds, confirming that data transmission delays were minimal and within acceptable limits for IoT-based systems.

Moreover, the Over-The-Air (OTA) firmware update feature was successfully implemented, enabling the device to perform remote firmware upgrades automatically via ThingsBoard without manual flashing. Overall, the proposed system achieved the project’s primary objectives by delivering an efficient, stable, and easily maintainable cloud-based IoT monitoring platform.

ACKNOWLEDGMENT

The authors would like to express their sincere gratitude to the Department of Instrumentation Engineering, Institut Teknologi Sepuluh Nopember (ITS), for providing continuous support during the development of this research. The authors also wish to extend special appreciation to Mr. Ahmad Radhy, S.Si., M.Si., for his invaluable guidance, constructive feedback, and encouragement throughout the preparation of this article and the implementation of the IoT monitoring system project.

REFERENCES

- [1] A. Al-Fuqaha, M. Guizani, M. Mohammadi, M. Aledhari, dan M. Ayyash, “Internet of Things: A survey on enabling technologies, protocols, and applications,” *IEEE Communications Surveys & Tutorials*, vol. 17, no. 4, hlm. 2347–2376, 2015.
- [2] L. Atzori, A. Iera, dan G. Morabito, “Understanding the Internet of Things: Definition, potentials, and societal role,” *Ad Hoc Networks*, vol. 56, hlm. 122–140, 2016.
- [3] L. O. Aghenta dan M. T. Iqbal, “Design and implementation of a low-cost, open source IoT-based SCADA system using ESP32 with OLED, ThingsBoard and MQTT protocol,” *AIMS Electronics and Electrical Engineering*, vol. 4, no. 1, hlm. 57–74, 2020.
- [4] S. El Jaouhari dan E. Bouvet, “Secure over-the-air updates for IoT: Survey, challenges, and discussions,” *Internet of Things*, vol. 18, hlm. 100508, 2022.
- [5] D. Hercog, T. Lerher, M. Truntiĉ, dan O. Težak, “Design and implementation of ESP32-based IoT devices,” *Sensors*, vol. 23, no. 15, hlm. 6739, 2023.
- [6] Y. A. Ahmad, T. S. Gunawan, H. Mansor, B. A. Hamida, A. F. Hishamudin, dan F. Arifin, “On the evaluation of DHT22 temperature sensor for IoT application,” *Proc. 8th Int. Conf. Computer and Communication Engineering (ICCCCE)*, hlm. 131–134, 2021.
- [7] D. Yulizar, S. Soekimo, N. Ananda, M. A. Prabowo, I. F. Putra Perdana, dan D. Aofany, “Performance analysis comparison of DHT11, DHT22, and DS18B20 temperature sensors,” *Proc. 2023 Int. Conf. on Computing and Applied Electronics*, Atlantis Press, 2023.
- [8] C.-Y. Park, S.-J. Lee, dan I.-G. Lee, “Secure and lightweight Over-the-Air update mechanism for Internet of Things,” *Electronics*, vol. 14, no. 8, hlm. 1583, 2025.

- [9] V. Malumbres, J. Saldana, G. Berné, dan J. Modrego, "Updates over the air via LoRa: Unicast and broadcast combination for boosting update speed," *Sensors*, vol. 24, no. 7, hlm. 2104, 2024.
- [10] I. Plauska, A. Liutkevičius, dan A. Janavičiūtė, "Performance evaluation of C/C++, MicroPython, Rust and TinyGo programming languages on ESP32 microcontroller," *Electronics*, vol. 12, no. 1, hlm. 143, 2023.
- [11] I. Culic, A. Vochescu, dan A. Radovici, "A low-latency optimization of a Rust-based secure operating system for embedded devices," *Sensors*, vol. 22, no. 22, hlm. 8700, 2022.
- [12] "Integrating Edge Computing and IoT for Real-Time Air and Water Quality Monitoring Systems," *Int. J. Environmental Sciences*, vol. 11, no. 17s, 2025.
- [13] J. Roostaei, dkk., "IoT-based edge computing (IoTEC) for improved environmental monitoring," *ScienceDirect*, 2023.
- [14] F. Peng, "Environment Monitoring based on the Integration of Edge Computing and Multimodal Sensors," *Applied and Computational Engineering*, vol. 113, hlm. 77–82, 2024.
- [15] N. Saha, M. M. Aulia, M. M. Rahman, dan M. S. A. Khan, "IoT-Driven Cloud-based Energy and Environment Monitoring System for Manufacturing Industry," *arXiv preprint*, 2024.
- [16] A. Shvaika, D. Shvaika, D. Landiak, dkk., "A distributed architecture for MQTT messaging: The case of TBMQ," *Journal of Big Data*, vol. 12, hlm. 224, 2025.
- [17] R. M. M. Montesclaros, J. E. B. Cruz, R. C. Parocha, dan E. Q. B. Macabebe, "MQTT-Based Power Consumption Monitoring with Usage Pattern Visualization Using UMAP for Smart Buildings," *Proc. Intelligent Computing Conf.*, Springer Nature, 2021.
- [18] B. M. M. El-Basioni, "A conceptual modeling approach of MQTT for IoT-based systems," *J. Electrical Systems and Information Technology*, vol. 11, Artikel 62, 2024.
- [19] P. Putri, M. A. Saragi, dan Y. A. E. H. Hasibuan, "A prototype IoT temperature and humidity monitoring ESP32-based via ThingSpeak," *INFOKUM: Jurnal Informatika dan Komputer*, vol. 13, no. 1, hlm. 15–21, 2024.
- [20] I. K. Wardani, A. N. Ichniarsyah, M. Telaumbanua, B. Priyonggo, R. Fil'aini, Z. Mufidah, dan D. A. Dewangga, "The feasibility study: Accuracy and precision of DHT22 in measuring the temperature and humidity in the greenhouse," *IOP Conf. Ser.: Earth and Environmental Science*, vol. 1290, no. 1, hlm. 012032, 2024.
- [21] J. J. Puthiyidam dan S. Joseph, "Internet of things network performance: Impact of message and client sizes and reliability levels," *ECTI Trans. on Electrical Engineering, Electronics, and Communications*, vol. 22, no. 1, 2024.
- [22] B. Amirkhanov, G. Amirkhanova, M. Kunelbayev, S. Adilzhanova, dan M. Tokhtassyn, "Evaluating HTTP, MQTT over TCP and MQTT over WebSocket for digital twin applications: A comparative analysis on latency, stability, and integration," *Int. J. Innovative Research and Scientific Studies*, vol. 8, no. 1, hlm. 679–694, 2025.
- [23] F. Palmese, A. E. C. Redondi, dan M. Cesana, "Adaptive Quality of Service Control for MQTT-SN," *Sensors*, vol. 22, no. 22, hlm. 8852, 2022.
- [24] W. Wei, J. Banerjee, S. Islam, C. Pan, dan M. Xie, "Energy-aware Incremental OTA Update for Flash-based Batteryless IoT Devices," *arXiv preprint*, arXiv:2406.12189, 2023.